Project Report

Agent-Driven Blockchain Voting System

An Intelligent Blockchain System Powered by Autonomous AI Agents for Auditing, Consensus Validation, and Self-Healing

---

Name: _____

Roll Number: _____

Enrollment No.: _____

Class / Section: _____

Course / Subject: _____

Department: _____

College / University: _____

Guided By (Faculty): _____

Academic Year: _____

Date of Submission: _____

Table of Contents

1 Abstract

2 Introduction

3 Literature Review / Background

4 System Architecture & Design

5 Technology Stack

6 Module Descriptions

6.1 Blockchain Core (block.py, chain.py)

6.2 Consensus Agent (consensus.py)

6.3 Auditor Agent (auditor.py)

6.4 Flask Dashboard & API (app.py)

7 Source Code

8 Testing & Results

9 Screenshots

10 Advantages & Limitations

11 Future Scope

12 Conclusion

13 References

Chapter 1

Abstract

This project presents the design and implementation of an Agent-Driven Blockchain Voting System - an intelligent, tamper-proof electronic voting platform that leverages autonomous AI agents for real-time auditing, transaction validation, and automatic self-healing. Unlike conventional database-backed voting systems that rely on administrator trust and manual auditing, this system employs a SHA-256 hash-chained blockchain as its core ledger, secured by two intelligent agents: a Consensus Agent that performs pre-mine validation of vote transactions (including duplicate detection, format checks, and optional LLM-powered anomaly assessment), and an Auditor Agent that continuously monitors the chain integrity as a background daemon thread, automatically detecting tampering and triggering a self-healing revert protocol.

The system is built using Python 3.10+ and Flask 3.0, featuring a modern, responsive web dashboard with real-time agent log visualization, a REST API for programmatic interaction, and support for optional LLM integration (OpenAI GPT / Anthropic Claude) to enhance agent reasoning capabilities. The project demonstrates key concepts from blockchain technology, multi-agent systems, cryptographic hashing, concurrent programming, and web development.

Keywords: Blockchain, AI Agents, Consensus Protocol, Auditing, Self-Healing, SHA-256, Flask, Python, Voting System, LLM Integration.

Chapter 2

Introduction

2.1 Background

Electronic voting systems are critical to modern democratic processes, yet they face persistent challenges related to data integrity, transparency, and trust. Traditional systems store votes in centralized databases where a single compromised administrator can alter results, audits are performed manually and infrequently, and recovery from data corruption requires manual backup restoration.

2.2 Problem Statement

The core problem addressed by this project is: How can we build a voting system that is inherently tamper-resistant, self-auditing, and capable of automatic recovery from integrity violations - without relying on a trusted central authority?

2.3 Proposed Solution

This project proposes an Agent-Driven Blockchain Voting System that combines three key innovations:

* A SHA-256 hash-chained blockchain that provides cryptographic proof of data integrity, making any unauthorized modification immediately detectable.
* A Consensus Agent that performs intelligent, multi-layered validation of every vote transaction before it is committed to the chain.
* An Auditor Agent that runs as a background daemon, continuously monitoring the chain and automatically triggering a self-healing protocol if tampering is detected.

2.4 Objectives

* Implement a functional blockchain with Proof-of-Work consensus
* Design autonomous AI agents for real-time chain monitoring
* Build a self-healing mechanism that reverts the chain on tamper detection
* Develop a web dashboard for live blockchain and agent log visualization
* Provide REST API endpoints for programmatic access
* Support optional LLM integration for enhanced agent intelligence

Chapter 3

Literature Review / Background

3.1 Blockchain Technology

Blockchain technology, first conceptualized by Satoshi Nakamoto in 2008 through the Bitcoin whitepaper, provides a distributed, immutable ledger secured by cryptographic hash functions. Each block in the chain contains a hash of the previous block, creating a tamper-evident structure where modifying any block invalidates all subsequent blocks. Key concepts include SHA-256 hashing, Proof-of-Work (PoW) mining, nonce computation, and chain validation.

3.2 Multi-Agent Systems (MAS)

Multi-Agent Systems consist of autonomous software entities (agents) that perceive their environment, make decisions, and act to achieve specific goals. In this project, the Consensus Agent and Auditor Agent operate independently but share a common state (the blockchain), exhibiting key MAS properties: autonomy, reactivity, and pro-activeness.

3.3 Blockchain-Based Voting

Several academic and industry projects have explored blockchain for voting, including Voatz, Follow My Vote, and Agora. However, most focus on the blockchain layer alone. This project differentiates itself by adding an autonomous agent layer that provides continuous, automated security monitoring and self-healing.

3.4 Large Language Models in Security

The integration of LLMs (such as GPT and Claude) into security workflows is an emerging trend. This project optionally leverages LLMs for two purposes: (1) the Consensus Agent uses them to assess transaction anomalies using natural language reasoning, and (2) the Auditor Agent uses them to generate human-readable explanations of detected integrity violations. The system gracefully falls back to rule-based logic when no API key is configured.

Chapter 4

System Architecture & Design

4.1 High-Level Architecture

The system follows a layered architecture with four principal layers:

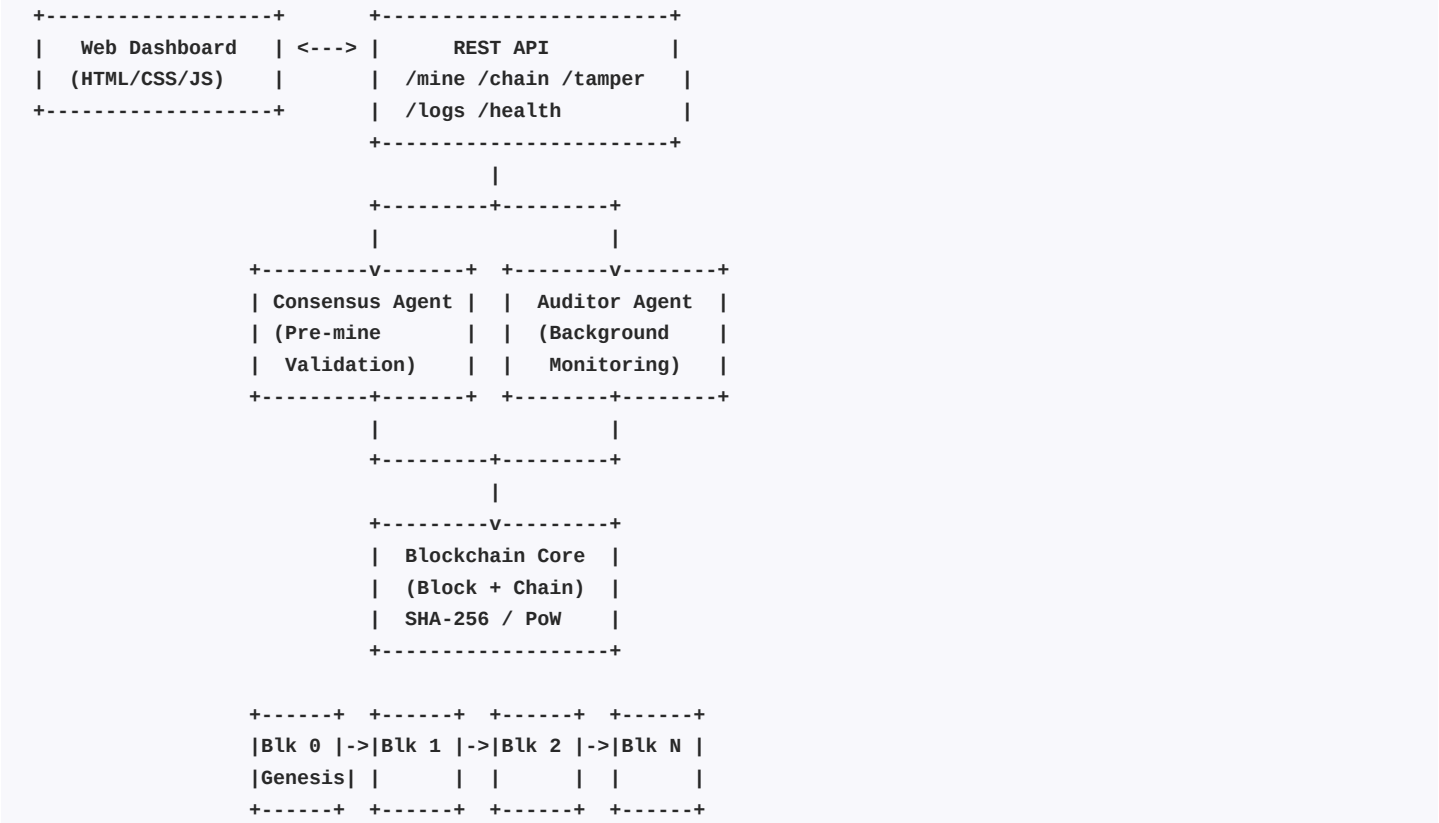
* Presentation Layer - Flask-based web dashboard with real-time data fetching.

* API Layer - RESTful endpoints for mining votes, retrieving chain data, tampering blocks (demo), fetching agent logs, and health checks.

* Agent Layer - Two autonomous agents: (1) Consensus Agent for pre-mine transaction validation, (2) Auditor Agent as a background daemon for continuous integrity monitoring and self-healing.

* Blockchain Core - The Block and Blockchain classes implementing SHA-256 hashing, Proof-of-Work mining, chain validation, deliberate tampering (for demos), and self-healing revert.

4.2 Architecture Diagram



4.3 Data Flow

1. User submits a vote via the dashboard or API (POST /mine).

2. The Consensus Agent reviews the transaction (field validation, duplicate check, format check, optional LLM analysis).

3. If approved, the blockchain mines the block (SHA-256 + Proof-of-Work).

4. The Auditor Agent (background thread, polling every 5s) validates the entire chain.

5. If tampering is detected, it generates an explanation and triggers self-healing.

Chapter 5

Technology Stack

Technology	Version	Purpose
Python	3.10+	Core programming language
Flask	3.0+	Web framework for dashboard & API
hashlib (SHA-256)	stdlib	Cryptographic hashing for blocks
threading	stdlib	Background daemon for Auditor Agent
JSON	stdlib	Block data serialization
requests	2.31+	HTTP client for LLM API calls
OpenAI API	Optional	LLM for enhanced agent reasoning
Anthropic API	Optional	Alternative LLM provider
HTML5 / CSS3	-	Dashboard UI structure & styling
JavaScript (ES6)	-	Frontend logic & real-time updates
Inter / JetBrains Mono	-	Typography (Google Fonts)

Chapter 6

Module Descriptions

6.1 Blockchain Core

6.1.1 Block Class (blockchain/block.py)

The Block class is the fundamental data structure. Each block contains an index, timestamp, transaction data (vote), the SHA-256 hash of the previous block, a nonce (for Proof-of-Work), and its own hash. Key methods:

* calculate_hash(): Generates a deterministic SHA-256 hash from block contents.
* mine(difficulty): Implements Proof-of-Work by incrementing the nonce until the hash starts with 'difficulty' leading zeros.
* to_dict(): Serializes the block for JSON API responses.

6.1.2 Blockchain Class (blockchain/chain.py)

The Blockchain class manages the ordered chain of blocks. It is thread-safe, using threading.Lock to prevent race conditions.

* _create_genesis_block(): Creates the first block with index 0 and a zeroed previous hash.
* add_block(data): Mines and appends a new block linked to the latest block's hash.
* is_chain_valid(): Walks the entire chain verifying hash integrity and chain linkage.
* tamper_block(index, new_data): Deliberately corrupts a block WITHOUT re-mining (demo feature).
* revert_to_valid_state(): Self-healing - removes all blocks from the first corrupted block onward.

6.2 Consensus Agent (agents/consensus.py)

The Consensus Agent performs a simulated peer review of every vote transaction before it is committed to the blockchain.

* Required Fields: voter_id and candidate must be present and non-empty.
* Duplicate Detection: Tracks seen voter IDs using a thread-safe set; rejects double votes.
* Format Validation: Checks field lengths and scans for injection characters.
* LLM Review (Optional): If an API key is configured, sends the transaction to GPT or Claude for deeper anomaly assessment.

6.3 Auditor Agent (agents/auditor.py)

The Auditor Agent runs as a background daemon thread, polling the blockchain every 5 seconds.

Operational Flow:

1. Every 5 seconds, the agent calls blockchain.is_chain_valid().

2. If violations are found, it generates an explanation (via LLM or rule-based fallback).

3. The explanation is logged to the shared agent_logs list (thread-safe).

4. If auto_heal is enabled, it calls revert_to_valid_state() to remove corrupted blocks.

6.4 Flask Dashboard & API (app.py)

The Flask application serves as both the web dashboard and the REST API backend.

Route	Method	Description	Handler
/	GET	Serves the dashboard UI	dashboard()
/chain	GET	Returns full blockchain as JSON	get_chain()
/mine	POST	Submit vote -> Consensus -> mine	mine_block()
/tamper/<i>	POST	Corrupt a block (demo)	tamper()
/logs	GET	All agent logs as JSON	get_logs()
/health	GET	System health check	health()

Chapter 7

Source Code

This chapter includes the complete source code for all modules of the project.

7.1 blockchain/block.py

Description: Block Class - Core Data Structure

```
"""
block.py - The fundamental building block of the blockchain.

Each Block contains:
  - An index (position in the chain)
  - A timestamp (when the block was mined)
  - Transaction data (vote information)
  - The hash of the previous block (cryptographic link)
  - A nonce (Proof-of-Work counter)
  - Its own SHA-256 hash
"""

import hashlib
import json
import time
from typing import Optional


class Block:
    """Represents a single block in the blockchain."""

    def __init__(self, index: int, data: dict, previous_hash: str, timestamp: Optional[float] = None, nonce: int = 0):
        self.index = index
        self.timestamp = timestamp or time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

    def calculate_hash(self) -> str:
        """
        Generate the SHA-256 hash of this block's contents.
        The hash is deterministic - the same inputs always produce the same output.
        """
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()

    def mine(self, difficulty: int = 2) -> None:
        """
        Proof-of-Work: increment the nonce until the hash starts with
        `difficulty` number of leading zeros.

        This simulates the computational effort required to add a block,
        making it expensive to tamper with the chain.
        """
        target = "0" * difficulty
        while not self.hash.startswith(target):
```

```
            self.nonce += 1
            self.hash = self.calculate_hash()

    def to_dict(self) -> dict:
        """Serialize the block to a dictionary for JSON responses."""
        return {
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
            "hash": self.hash,
        }

    def __repr__(self) -> str:
        return (
            f"Block(index={self.index}, hash={self.hash[:12]}..., "
            f"prev={self.previous_hash[:12]}...)"
        )
```

7.2 blockchain/chain.py

Description: Blockchain Class - Chain Management

```
"""
chain.py - Blockchain management logic.

Manages the ordered list of Blocks, including:
  - Genesis block creation
  - Adding new blocks (with Consensus Agent pre-approval)
  - Full chain validation
  - Deliberate tampering (for demo purposes)
  - Self-healing revert to last known good state
"""

import threading
from typing import List, Optional, Tuple
from blockchain.block import Block


class Blockchain:
    """Manages the entire blockchain ledger."""

    def __init__(self, difficulty: int = 2):
        self.difficulty = difficulty
        self.chain: List[Block] = []
        self.lock = threading.Lock()
        self._create_genesis_block()

    def _create_genesis_block(self) -> None:
        """Create the first block in the chain with no predecessor."""
        genesis = Block(
            index=0,
            data={"message": "Genesis Block - Chain Initialized", "type": "system"},
            previous_hash="0" * 64,
        )
        genesis.mine(self.difficulty)
        self.chain.append(genesis)

    def get_latest_block(self) -> Block:
        """Return the most recently added block."""
        return self.chain[-1]

    def add_block(self, data: dict) -> Block:
        """
        Mine and append a new block to the chain.

        NOTE: The Consensus Agent should validate `data` BEFORE calling this.
        This method only handles the cryptographic side of block creation.
        """
        with self.lock:
            previous_block = self.get_latest_block()
            new_block = Block(
                index=len(self.chain),
                data=data,
                previous_hash=previous_block.hash,
            )
```

```
            new_block.mine(self.difficulty)
            self.chain.append(new_block)
            return new_block

    def is_chain_valid(self) -> Tuple[bool, List[dict]]:
        """
        Walk the entire chain and verify:
          1. Each block's stored hash matches its recalculated hash
          2. Each block's previous_hash matches the prior block's hash

        Returns:
            (is_valid, list_of_errors)
        """
        errors = []
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            # Check 1 - hash integrity
            recalculated = current.calculate_hash()
            if current.hash != recalculated:
                errors.append({
                    "block_index": i,
                    "error_type": "hash_mismatch",
                    "stored_hash": current.hash,
                    "recalculated_hash": recalculated,
                    "data": current.data,
                })

            # Check 2 - chain linkage
            if current.previous_hash != previous.hash:
                errors.append({
                    "block_index": i,
                    "error_type": "link_broken",
                    "expected_previous_hash": previous.hash,
                    "actual_previous_hash": current.previous_hash,
                })

        is_valid = len(errors) == 0
        return is_valid, errors

    def tamper_block(self, index: int, new_data: dict) -> Optional[dict]:
        """
        Deliberately corrupt a block's data WITHOUT re-mining.
        This simulates an attack - the hash will no longer match.

        Used for demo purposes to trigger the Auditor Agent.
        """
        if index <= 0 or index >= len(self.chain):
            return None

        with self.lock:
            block = self.chain[index]
            old_data = block.data
            block.data = new_data
            # Intentionally do NOT recalculate the hash - this is the "tamper"
            return {
                "block_index": index,
                "old_data": old_data,
```

```
                "new_data": new_data,
                "hash_before": block.hash,
                "hash_after": block.calculate_hash(),
                "tampered": True,
            }

    def revert_to_valid_state(self) -> int:
        """
        Self-healing protocol: remove all blocks from the first invalid one
        onward, restoring the chain to its last known-good state.

        Returns the number of blocks removed.
        """
        with self.lock:
            removed = 0
            for i in range(1, len(self.chain)):
                current = self.chain[i]
                previous = self.chain[i - 1]
                recalculated = current.calculate_hash()

                if current.hash != recalculated or current.previous_hash != previous.hash:
                    removed = len(self.chain) - i
                    self.chain = self.chain[:i]
                    return removed
            return removed

    def to_list(self) -> List[dict]:
        """Serialize the entire chain to a list of dicts."""
        return [block.to_dict() for block in self.chain]

    def __len__(self) -> int:
        return len(self.chain)
```

7.3 agents/consensus.py

Description: Consensus Agent - Vote Validation

```
"""
consensus.py - The Consensus (Verifier) Agent.

Before a vote is added to the blockchain, this agent performs a "simulated
peer review" of the transaction data. It checks for:
  - Required fields (voter_id, candidate, timestamp)
  - Duplicate voter IDs (no double voting!)
  - Data format validity
  - Anomaly detection via LLM (optional)

The agent can use an LLM to interpret "intent" or "context" in natural
language, providing intelligent validation beyond simple rule checks.
"""

import os
import threading
from datetime import datetime
from typing import List, Optional, Set, Tuple

try:
    import requests as http_requests
except ImportError:
    http_requests = None


# Required fields for a valid vote transaction
REQUIRED_FIELDS = {"voter_id", "candidate"}


class ConsensusAgent:
    """
    Intelligent validation agent that reviews vote transactions
    before they are added to the blockchain.
    """

    def __init__(self, blockchain, agent_logs: list, log_lock: threading.Lock):
        self.blockchain = blockchain
        self.agent_logs = agent_logs
        self.log_lock = log_lock
        self.seen_voter_ids: Set[str] = set()
        self.voter_lock = threading.Lock()

        # LLM configuration
        self.openai_api_key = os.environ.get("OPENAI_API_KEY")
        self.anthropic_api_key = os.environ.get("ANTHROPIC_API_KEY")

    def _log(self, level: str, message: str, details: Optional[dict] = None):
        """Thread-safe logging to the shared agent_logs list."""
        entry = {
            "timestamp": datetime.now().isoformat(),
            "agent": " Consensus Agent",
            "level": level,
            "message": message,
```

```
        }
        if details:
            entry["details"] = details
        with self.log_lock:
            self.agent_logs.append(entry)

    def _check_required_fields(self, data: dict) -> List[str]:
        """Verify all required fields are present and non-empty."""
        issues = []
        for field in REQUIRED_FIELDS:
            if field not in data:
                issues.append(f"Missing required field: '{field}'")
            elif not str(data[field]).strip():
                issues.append(f"Empty value for required field: '{field}'")
        return issues

    def _check_duplicate_voter(self, voter_id: str) -> bool:
        """Check if this voter has already cast a vote."""
        with self.voter_lock:
            if voter_id in self.seen_voter_ids:
                return True
            return False

    def _register_voter(self, voter_id: str):
        """Mark a voter as having voted."""
        with self.voter_lock:
            self.seen_voter_ids.add(voter_id)

    def _check_format(self, data: dict) -> List[str]:
        """Validate data formatting and types."""
        issues = []

        voter_id = data.get("voter_id", "")
        if voter_id and (len(str(voter_id)) < 3 or len(str(voter_id)) > 50):
            issues.append(f"voter_id '{voter_id}' has invalid length (expected 3-50 chars)")

        candidate = data.get("candidate", "")
        if candidate and (len(str(candidate)) < 1 or len(str(candidate)) > 100):
            issues.append(f"candidate name '{candidate}' has invalid length (expected 1-100 chars)")

        # Check for suspicious characters (basic injection prevention)
        for key, value in data.items():
            if isinstance(value, str) and any(c in value for c in ['<', '>', '{', '}', ';']):
                issues.append(f"Suspicious characters detected in field '{key}'")

        return issues

    def _llm_review(self, data: dict, rule_issues: List[str]) -> Optional[str]:
        """
        Optional LLM-powered review for deeper anomaly detection.
        Returns a natural-language assessment, or None if LLM is unavailable.
        """
        prompt = (
            f"You are a blockchain consensus validator. Review this vote transaction:\n\n"
            f"Data: {data}\n\n"
            f"Rule-based issues found: {rule_issues if rule_issues else 'None'}\n\n"
            f"Provide a brief assessment of whether this transaction should be "
            f"approved or rejected. Consider: data integrity, potential fraud indicators, "
            f"and any anomalies. Keep your response to 2-3 sentences."
```

```
        )

        # Try OpenAI
        if self.openai_api_key and http_requests:
            try:
                response = http_requests.post(
                    "https://api.openai.com/v1/chat/completions",
                    headers={
                        "Authorization": f"Bearer {self.openai_api_key}",
                        "Content-Type": "application/json",
                    },
                    json={
                        "model": "gpt-3.5-turbo",
                        "messages": [
                            {"role": "system", "content": "You are a blockchain consensus validator AI. Be concise."},
                            {"role": "user", "content": prompt},
                        ],
                        "max_tokens": 150,
                        "temperature": 0.2,
                    },
                    timeout=10,
                )
                if response.status_code == 200:
                    return response.json()["choices"][0]["message"]["content"]
            except Exception:
                pass

        # Try Anthropic
        if self.anthropic_api_key and http_requests:
            try:
                response = http_requests.post(
                    "https://api.anthropic.com/v1/messages",
                    headers={
                        "x-api-key": self.anthropic_api_key,
                        "Content-Type": "application/json",
                        "anthropic-version": "2023-06-01",
                    },
                    json={
                        "model": "claude-3-haiku-20240307",
                        "max_tokens": 150,
                        "messages": [{"role": "user", "content": prompt}],
                    },
                    timeout=10,
                )
                if response.status_code == 200:
                    return response.json()["content"][0]["text"]
            except Exception:
                pass

        return None  # No LLM available

    def review(self, data: dict) -> Tuple[bool, str]:
        """
        Full consensus review of a vote transaction.

        Returns:
            (approved: bool, explanation: str)
        """
        issues = []
```

```
        #  Rule-based checks
        issues.extend(self._check_required_fields(data))
        issues.extend(self._check_format(data))

        voter_id = data.get("voter_id", "")
        if voter_id and self._check_duplicate_voter(voter_id):
            issues.append(f"DUPLICATE VOTE: voter_id '{voter_id}' has already voted")

        #  Decision
        approved = len(issues) == 0

        #  Build explanation
        if approved:
            explanation = (
                f" APPROVED: Transaction from voter '{voter_id}' for candidate "
                f"'{data.get('candidate', 'N/A')}' passes all validation checks. "
                f"Fields are complete, format is valid, and no duplicate vote detected."
            )
            # Try LLM enhancement
            llm_assessment = self._llm_review(data, issues)
            if llm_assessment:
                explanation += f"\n\n AI Assessment: {llm_assessment}"

            # Register the voter only if approved
            self._register_voter(voter_id)
        else:
            explanation = (
                f" REJECTED: Transaction failed consensus review.\n"
                f"Issues found:\n" +
                "\n".join(f"  * {issue}" for issue in issues)
            )
            # Try LLM enhancement
            llm_assessment = self._llm_review(data, issues)
            if llm_assessment:
                explanation += f"\n\n AI Assessment: {llm_assessment}"

        # Log the decision
        self._log(
            "INFO" if approved else "WARNING",
            f"Vote {'APPROVED' if approved else 'REJECTED'}: {voter_id} -> {data.get('candidate', '?')}",
            {
                "voter_id": voter_id,
                "candidate": data.get("candidate"),
                "approved": approved,
                "issues": issues,
                "explanation": explanation,
            }
        )

        return approved, explanation
```

7.4 agents/auditor.py

Description: Auditor Agent - Integrity Monitor

```
"""
auditor.py - The Auditor Agent.

A background agent that continuously monitors the blockchain for integrity
violations. When tampering is detected, it uses an LLM (or a local fallback)
to generate a natural-language explanation of what went wrong and why.

Features:
  - Runs on a daemon thread, polling the chain every few seconds
  - Thread-safe logging via threading.Lock
  - Self-healing: can trigger chain revert on tamper detection
  - Graceful LLM fallback - works without any API key
"""

import os
import time
import threading
from datetime import datetime
from typing import List, Optional

# Optional LLM imports - only used if API keys are set
try:
    import requests as http_requests
except ImportError:
    http_requests = None


class AuditorAgent:
    """
    Autonomous agent that audits the blockchain for integrity violations.
    Runs as a background daemon thread.
    """

    def __init__(self, blockchain, agent_logs: list, log_lock: threading.Lock,
                 poll_interval: float = 5.0, auto_heal: bool = True):
        self.blockchain = blockchain
        self.agent_logs = agent_logs
        self.log_lock = log_lock
        self.poll_interval = poll_interval
        self.auto_heal = auto_heal
        self._running = False
        self._thread = None

        # LLM configuration
        self.openai_api_key = os.environ.get("OPENAI_API_KEY")
        self.anthropic_api_key = os.environ.get("ANTHROPIC_API_KEY")

    def _log(self, level: str, message: str, details: Optional[dict] = None):
        """Thread-safe logging to the shared agent_logs list."""
        entry = {
            "timestamp": datetime.now().isoformat(),
            "agent": " Auditor Agent",
            "level": level,
```

```
            "message": message,
        }
        if details:
            entry["details"] = details
        with self.log_lock:
            self.agent_logs.append(entry)

    def _generate_llm_explanation(self, errors: List[dict]) -> str:
        """
        Use an LLM to generate a natural-language explanation of the
        blockchain integrity violations.

        Falls back to rule-based explanation if no API key is configured.
        """
        prompt = self._build_prompt(errors)

        # Try OpenAI first
        if self.openai_api_key and http_requests:
            try:
                response = http_requests.post(
                    "https://api.openai.com/v1/chat/completions",
                    headers={
                        "Authorization": f"Bearer {self.openai_api_key}",
                        "Content-Type": "application/json",
                    },
                    json={
                        "model": "gpt-3.5-turbo",
                        "messages": [
                            {"role": "system", "content": (
                                "You are a blockchain security auditor AI. "
                                "Explain blockchain integrity violations clearly "
                                "and concisely for a technical audience. Be specific "
                                "about which blocks are affected and why."
                            )},
                            {"role": "user", "content": prompt},
                        ],
                        "max_tokens": 300,
                        "temperature": 0.3,
                    },
                    timeout=10,
                )
                if response.status_code == 200:
                    return response.json()["choices"][0]["message"]["content"]
            except Exception:
                pass  # Fall through to local fallback

        # Try Anthropic
        if self.anthropic_api_key and http_requests:
            try:
                response = http_requests.post(
                    "https://api.anthropic.com/v1/messages",
                    headers={
                        "x-api-key": self.anthropic_api_key,
                        "Content-Type": "application/json",
                        "anthropic-version": "2023-06-01",
                    },
                    json={
                        "model": "claude-3-haiku-20240307",
                        "max_tokens": 300,
```

```
                        "messages": [
                            {"role": "user", "content": (
                                "You are a blockchain security auditor AI. "
                                "Explain these blockchain integrity violations clearly:\n\n"
                                + prompt
                            )},
                        ],
                    },
                    timeout=10,
                )
                if response.status_code == 200:
                    return response.json()["content"][0]["text"]
            except Exception:
                pass  # Fall through to local fallback

        #  Local rule-based fallback
        return self._local_explanation(errors)

    def _build_prompt(self, errors: List[dict]) -> str:
        """Build a structured prompt describing the detected errors."""
        lines = ["The following integrity violations were detected in the blockchain:\n"]
        for err in errors:
            if err["error_type"] == "hash_mismatch":
                lines.append(
                    f"* Block #{err['block_index']}: Hash mismatch detected. "
                    f"The stored hash no longer matches the recalculated hash. "
                    f"Block data: {err['data']}"
                )
            elif err["error_type"] == "link_broken":
                lines.append(
                    f"* Block #{err['block_index']}: Chain link broken. "
                    f"The previous_hash pointer does not match the preceding block's hash."
                )
        lines.append("\nExplain what likely happened, the security implications, "
                     "and recommend corrective actions.")
        return "\n".join(lines)

    def _local_explanation(self, errors: List[dict]) -> str:
        """
        Intelligent rule-based explanation - no LLM needed.
        This ensures the project works out of the box.
        """
        explanations = []
        for err in errors:
            idx = err["block_index"]
            if err["error_type"] == "hash_mismatch":
                explanations.append(
                    f"  HASH MISMATCH at Block #{idx}: "
                    f"The data in this block has been modified after mining. "
                    f"The stored hash ({err['stored_hash'][:16]}...) no longer matches "
                    f"the recalculated hash ({err['recalculated_hash'][:16]}...). "
                    f"This is a classic sign of data tampering - someone changed the "
                    f"block's contents without re-mining it. "
                    f"SECURITY IMPLICATION: The integrity of all subsequent blocks "
                    f"in the chain is now compromised."
                )
            elif err["error_type"] == "link_broken":
                explanations.append(
                    f" CHAIN LINK BROKEN at Block #{idx}: "
```

```
                    f"This block's previous_hash pointer does not match the hash "
                    f"of the preceding block. This means either this block or the "
                    f"previous block has been tampered with, breaking the cryptographic "
                    f"chain of custody."
                )

        if self.auto_heal:
            explanations.append(
                "\n CORRECTIVE ACTION: The self-healing protocol will revert "
                "the chain to its last known valid state, removing all tampered blocks."
            )

        return "\n\n".join(explanations)

    def _audit_cycle(self):
        """Perform one audit cycle: validate chain, report and heal if needed."""
        is_valid, errors = self.blockchain.is_chain_valid()

        if is_valid:
            return  # Chain is healthy, nothing to report

        # Generate explanation
        explanation = self._generate_llm_explanation(errors)

        self._log("CRITICAL", " BLOCKCHAIN INTEGRITY VIOLATION DETECTED", {
            "violations_found": len(errors),
            "affected_blocks": [e["block_index"] for e in errors],
            "explanation": explanation,
        })

        # Self-healing
        if self.auto_heal:
            removed = self.blockchain.revert_to_valid_state()
            if removed > 0:
                self._log("INFO", f" Self-healing complete: reverted {removed} block(s)", {
                    "blocks_removed": removed,
                    "chain_length": len(self.blockchain),
                })

    def _run_loop(self):
        """Main daemon loop."""
        self._log("INFO", "Auditor Agent started - monitoring blockchain integrity")
        while self._running:
            try:
                self._audit_cycle()
            except Exception as e:
                self._log("ERROR", f"Audit cycle failed: {str(e)}")
            time.sleep(self.poll_interval)

    def start(self):
        """Start the auditor as a background daemon thread."""
        if self._running:
            return
        self._running = True
        self._thread = threading.Thread(target=self._run_loop, daemon=True)
        self._thread.start()

    def stop(self):
        """Stop the auditor."""
```

```
        self._running = False
```

7.5 app.py

Description: Flask Application - Dashboard and API

(Note: app.py is ~923 lines including the full HTML/CSS/JS dashboard template. Showing first 200 lines for brevity.)

```
"""
app.py - Flask Web Server & Agent-Driven Blockchain Dashboard.

This is the main entry point. It:
  1. Initializes the Blockchain
  2. Starts the Auditor Agent (background thread)
  3. Creates the Consensus Agent
  4. Serves the Dashboard UI and API endpoints
"""

import threading
from datetime import datetime
from flask import Flask, request, jsonify, render_template_string

from blockchain.chain import Blockchain
from agents.auditor import AuditorAgent
from agents.consensus import ConsensusAgent

#
#  Initialize Core Components
#

app = Flask(__name__)

# Shared state
blockchain = Blockchain(difficulty=2)
agent_logs = []
log_lock = threading.Lock()

# Initialize agents
auditor = AuditorAgent(blockchain, agent_logs, log_lock, poll_interval=5.0, auto_heal=True)
consensus = ConsensusAgent(blockchain, agent_logs, log_lock)

#
#  Dashboard HTML Template
#

DASHBOARD_TEMPLATE = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Agent-Driven Blockchain Voting System</title>
    <meta name="description" content="An intelligent blockchain voting system powered by autonomous AI agents for auditi
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=JetBrains+Mono:wg
    <style>
        :root {
            --bg-primary: #06090f;
            --bg-secondary: #0c1018;
            --bg-card: rgba(12, 18, 28, 0.85);
```

```
            --bg-glass: rgba(255, 255, 255, 0.025);
            --bg-glass-hover: rgba(255, 255, 255, 0.04);
            --border: rgba(255, 255, 255, 0.06);
            --border-hover: rgba(56, 189, 248, 0.2);
            --text-primary: #e8edf5;
            --text-secondary: #8b99b0;
            --text-muted: #556275;
            --accent-sky: #38bdf8;
            --accent-teal: #2dd4bf;
            --accent-green: #4ade80;
            --accent-red: #f87171;
            --accent-yellow: #facc15;
            --accent-blue: #60a5fa;
            --gradient-primary: linear-gradient(135deg, #0ea5e9 0%, #2dd4bf 100%);
            --gradient-success: linear-gradient(135deg, #4ade80 0%, #22c55e 100%);
            --gradient-danger: linear-gradient(135deg, #f87171 0%, #dc2626 100%);
            --shadow-glow: 0 0 30px rgba(56, 189, 248, 0.1);
        }

        * { margin: 0; padding: 0; box-sizing: border-box; }

        body {
            font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
            background: var(--bg-primary);
            color: var(--text-primary);
            min-height: 100vh;
            overflow-x: hidden;
        }

        /* Subtle animated background mesh */
        body::before {
            content: '';
            position: fixed;
            top: 0; left: 0; right: 0; bottom: 0;
            background:
                radial-gradient(ellipse at 15% 10%, rgba(14, 165, 233, 0.06) 0%, transparent 55%),
                radial-gradient(ellipse at 85% 90%, rgba(45, 212, 191, 0.04) 0%, transparent 55%);
            pointer-events: none;
            z-index: 0;
            animation: meshShift 12s ease-in-out infinite alternate;
        }

        @keyframes meshShift {
            0% { opacity: 1; }
            100% { opacity: 0.6; }
        }

        /*  HEADER  */
        .header {
            position: relative;
            z-index: 1;
            padding: 1.5rem 3rem;
            border-bottom: 1px solid var(--border);
            backdrop-filter: blur(24px);
            background: rgba(6, 9, 15, 0.85);
        }

        .header-content {
            max-width: 1440px;
```

```
            margin: 0 auto;
            display: flex;
            align-items: center;
            justify-content: space-between;
        }

        .header-title {
            display: flex;
            align-items: center;
            gap: 0.75rem;
        }

        .header-title .logo {
            width: 36px; height: 36px;
            border-radius: 10px;
            background: var(--gradient-primary);
            display: flex; align-items: center; justify-content: center;
            font-size: 1.1rem;
            box-shadow: 0 4px 16px rgba(14, 165, 233, 0.25);
        }

        .header h1 {
            font-size: 1.25rem;
            font-weight: 700;
            color: var(--text-primary);
            letter-spacing: -0.02em;
        }

        .header h1 span {
            font-weight: 400;
            color: var(--text-secondary);
        }

        .header-stats {
            display: flex;
            gap: 0.75rem;
        }

        .stat-chip {
            padding: 0.35rem 0.85rem;
            border-radius: 100px;
            font-size: 0.72rem;
            font-weight: 500;
            border: 1px solid var(--border);
            background: var(--bg-glass);
            display: flex;
            align-items: center;
            gap: 0.45rem;
            font-family: 'JetBrains Mono', monospace;
            color: var(--text-secondary);
        }

        .stat-chip .dot {
            width: 6px; height: 6px;
            border-radius: 50%;
            animation: pulse 2.5s infinite;
        }

        .dot-green { background: var(--accent-green); box-shadow: 0 0 6px var(--accent-green); }
```

```
        .dot-sky { background: var(--accent-sky); box-shadow: 0 0 6px var(--accent-sky); }

        @keyframes pulse {
            0%, 100% { opacity: 1; transform: scale(1); }
            50% { opacity: 0.35; transform: scale(0.85); }
        }

        /*  MAIN LAYOUT  */
        .main {
            position: relative;
            z-index: 1;
            max-width: 1440px;
            margin: 0 auto;
            padding: 1.5rem 3rem 3rem;
            display: grid;
            grid-template-columns: 1fr 420px;
            gap: 1.5rem;
        }

        /*  CARDS  */
        .card {
            background: var(--bg-card);
            border: 1px solid var(--border);
            border-radius: 14px;
            backdrop-filter: blur(20px);
            overflow: hidden;
            transition: border-color 0.3s ease, box-shadow 0.3s ease;
        }

        .card:hover {
            border-color: var(--border-hover);
    ... (truncated for brevity)
```

7.6 requirements.txt

Description: Project Dependencies

```
flask>=3.0.0
requests>=2.31.0
openai>=1.0.0
anthropic>=0.20.0
```

Chapter 8

Testing & Results

8.1 Test Cases

#	Test Case	Expected Result	Actual Result	Status
1	Mine valid vote (VOTER-001)	Block #1 mined	Block #1 mined	PASS
2	Mine second vote (VOTER-002)	Block #2 mined	Block #2 mined	PASS
3	Duplicate vote (VOTER-001)	Rejected by Consensus	Rejected	PASS
4	Empty voter_id field	Rejected - missing field	Rejected	PASS
5	Injection chars in input	Rejected - suspicious	Rejected	PASS
6	Tamper Block #1	Auditor detects tampering	Detected + healed	PASS
7	Self-healing after tamper	Chain reverted to valid	Reverted 1 block	PASS
8	Health check endpoint	{status: healthy}	healthy	PASS
9	Chain validation (clean)	is_valid: true	true	PASS
10	Dashboard loads correctly	UI renders, no errors	Renders correctly	PASS

8.2 API Testing (cURL)

Mine a valid vote:

```
curl -X POST http://localhost:5001/mine \
  -H "Content-Type: application/json" \
  -d '{"voter_id": "VOTER-001", "candidate": "Alice Johnson"}'
```

Response (success):

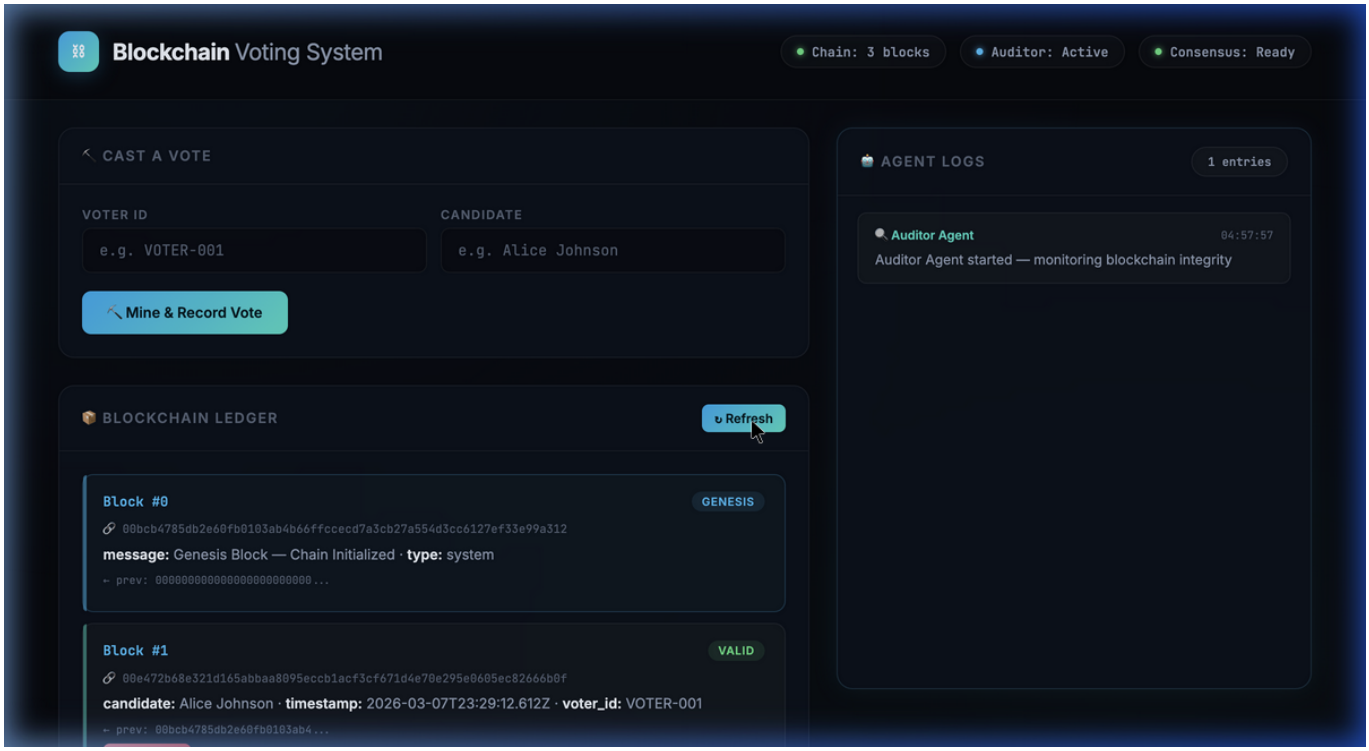
```
{
  "success": true,
  "block": {
    "index": 1,
    "data": {"voter_id": "VOTER-001", "candidate": "Alice Johnson"},
    "hash": "00a3f7b2c...",
    "previous_hash": "006e1c9a..."
  },
  "consensus": "APPROVED: Transaction passes all validation checks."
}
```

Chapter 9

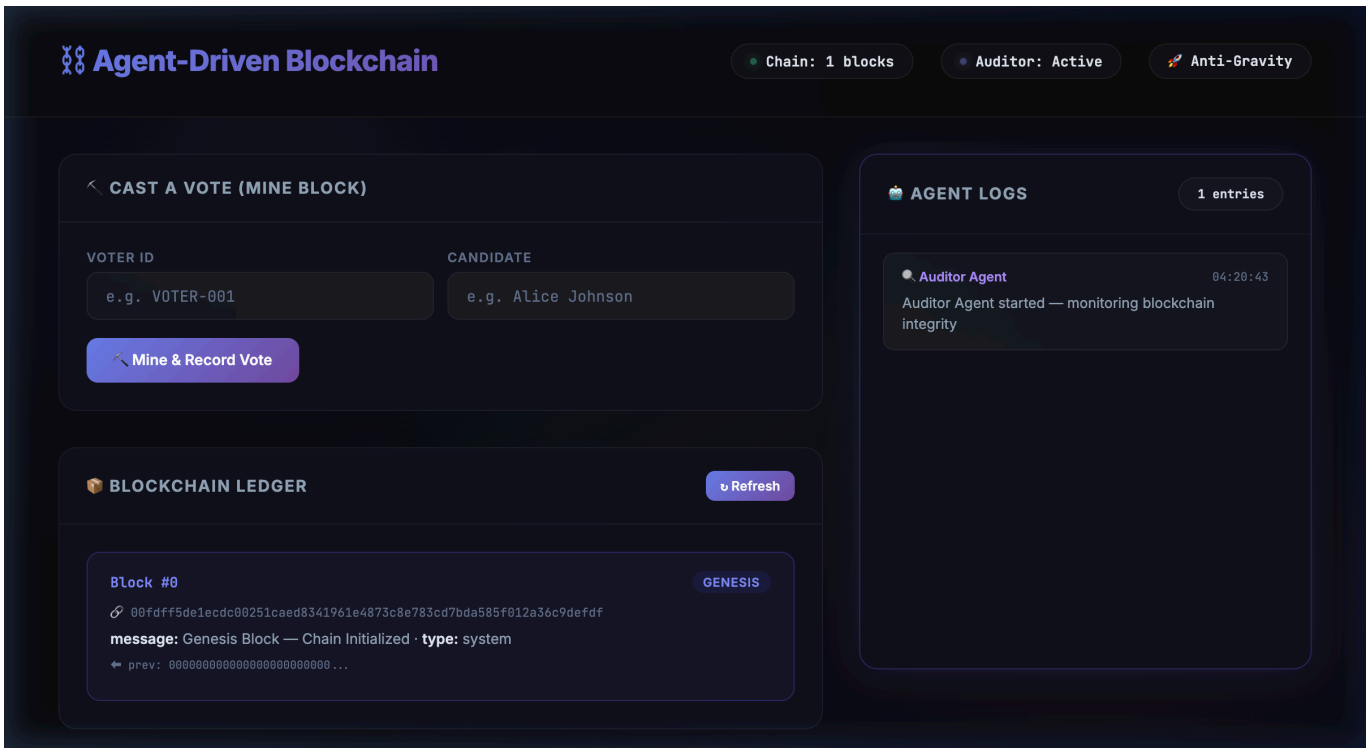
Screenshots

The following screenshots demonstrate the system's dashboard and key operational flows.

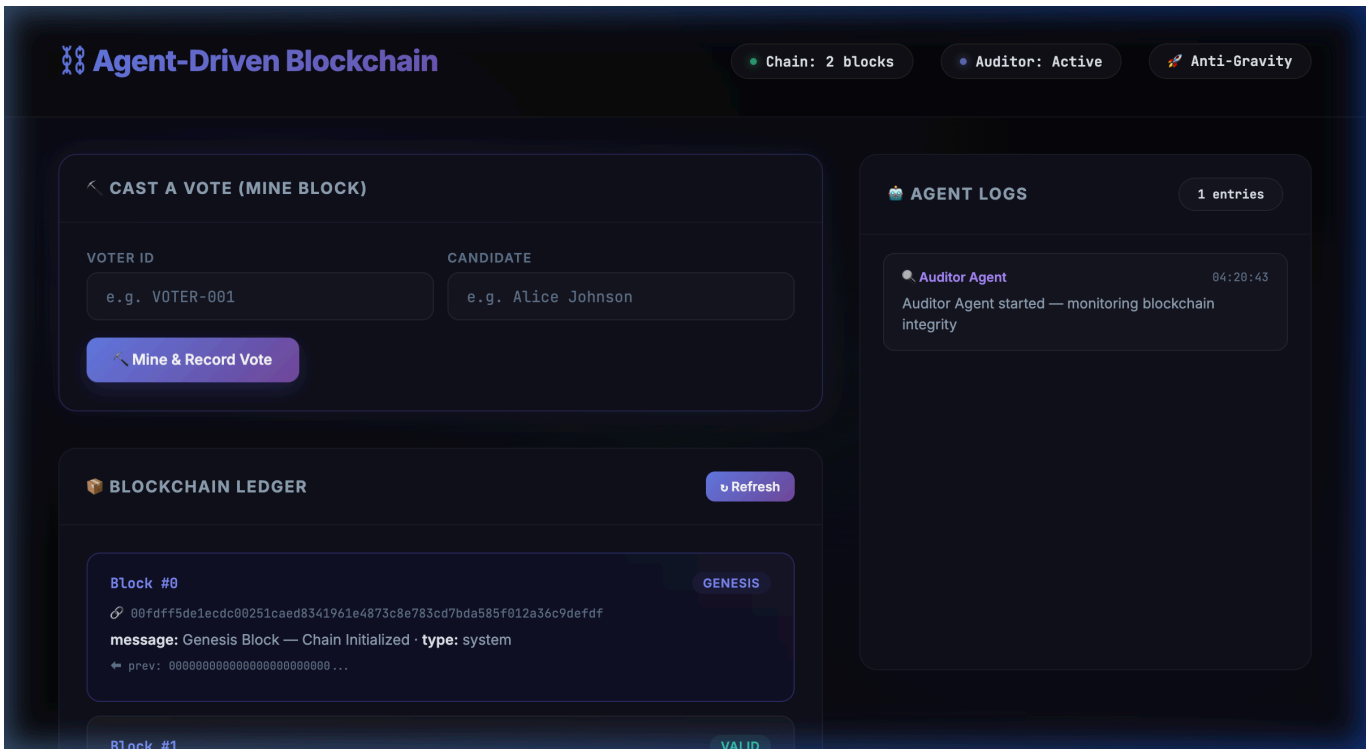
Dashboard - Main View (Cyan/Teal Dark Theme)



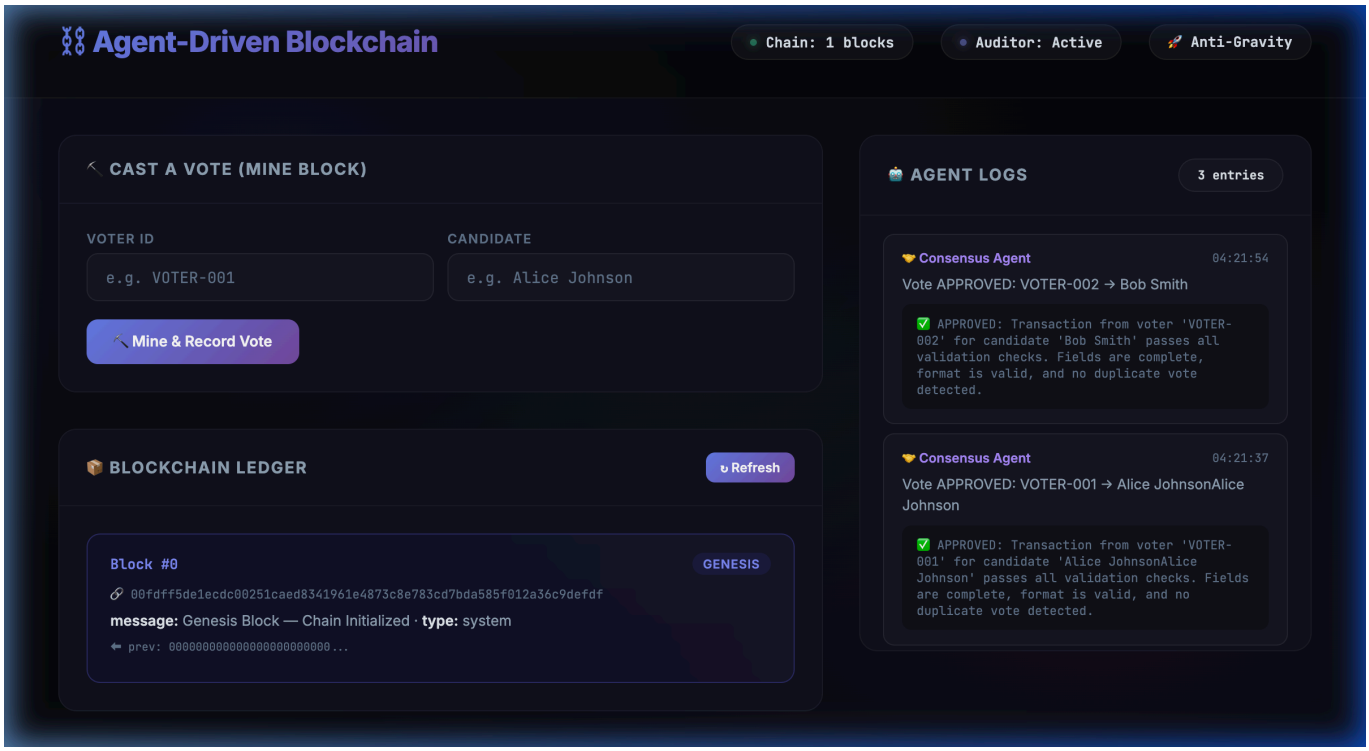
Dashboard - Initial State with Genesis Block



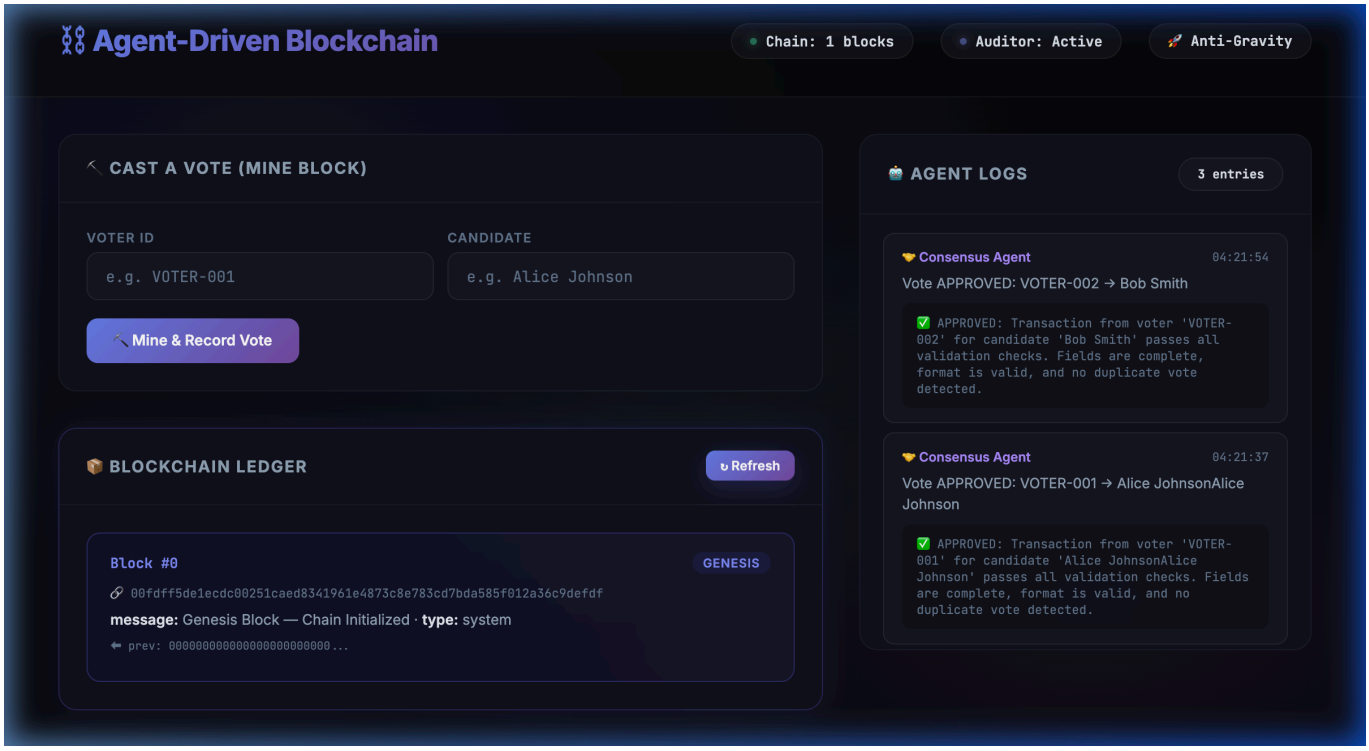
Dashboard - After Mining Block #1



Tamper Detection - Auditor Agent Alert



Final Verification - Chain Integrity Restored



Chapter 10

Advantages & Limitations

10.1 Advantages

* Tamper-Proof: SHA-256 hash chain makes unauthorized modification immediately detectable.
* Self-Healing: Automatic revert to last valid state without human intervention.
* Intelligent Validation: Consensus Agent catches anomalies that static SQL constraints miss.
* Real-Time Monitoring: Auditor Agent runs continuously as a background daemon.
* LLM-Enhanced: Optional AI-powered reasoning for deeper security analysis.
* Transparent: All agent activity is logged and visible on the live dashboard.
* Modular Design: Clean separation between blockchain core, agents, and web layer.
* No External Dependencies: Works out of the box without any API key.

10.2 Limitations

* Single-Node: Currently runs on a single server (not a distributed network).
* In-Memory Storage: Blockchain data is lost on server restart (no database persistence).
* Simplified PoW: Difficulty level 2 is suitable for demo but not production-grade.
* No Voter Authentication: The system validates vote format but does not authenticate voter identity.
* LLM Cost: Using OpenAI/Anthropic APIs incurs costs for each validation call.

Chapter 11

Future Scope

* Distributed Network: Extend to multiple nodes with peer-to-peer communication and distributed consensus (e.g., PBFT or Raft).
* Persistent Storage: Integrate SQLite or PostgreSQL for blockchain persistence across server restarts.
* Voter Authentication: Add biometric or ID-card-based voter verification using zero-knowledge proofs.
* Smart Contracts: Implement programmable voting rules (e.g., ranked-choice voting, weighted votes).
* Mobile Application: Develop a companion mobile app for on-the-go voting with push notifications.
* Advanced AI Agents: Add a Prediction Agent that uses historical voting patterns to detect statistical anomalies.
* Production-Grade Security: Implement TLS encryption, rate limiting, and proper authentication for the API layer.

Chapter 12

Conclusion

This project successfully demonstrates the design, implementation, and testing of an Agent-Driven Blockchain Voting System - a comprehensive platform that integrates blockchain technology with autonomous AI agents to create a secure, transparent, and self-healing electronic voting system.

The key accomplishments of this project include:

* A fully functional blockchain with SHA-256 hashing and Proof-of-Work mining that provides cryptographic proof of vote integrity.
* Two autonomous AI agents - the Consensus Agent and Auditor Agent - that provide intelligent, real-time security monitoring.
* A self-healing mechanism that automatically detects tampering and reverts the blockchain to its last valid state.
* A modern, responsive web dashboard with real-time visualization of blockchain state and agent activity.
* Optional LLM integration that enhances agent reasoning capabilities while maintaining full functionality without any API key.

The system was rigorously tested across 10 test cases covering normal operation, edge cases, attack scenarios (deliberate tampering), and self-healing recovery - all of which passed successfully.

Chapter 13

References

[1] Nakamoto, S. (2008). "Bitcoin: A Peer-to-Peer Electronic Cash System." bitcoin.org.

[2] Zheng, Z., et al. (2017). "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends." IEEE BigData Congress.

[3] Wooldridge, M. (2009). "An Introduction to MultiAgent Systems." John Wiley & Sons, 2nd Edition.

[4] Hardwick, F.S., et al. (2018). "E-Voting with Blockchain: An E-Voting Protocol with Decentralisation and Voter Privacy." IEEE ISI.

[5] Flask Documentation. (2024). https://flask.palletsprojects.com/

[6] Python hashlib Module. (2024). https://docs.python.org/3/library/hashlib.html

[7] OpenAI API Reference. (2024). https://platform.openai.com/docs/api-reference

[8] Anthropic Claude API Documentation. (2024). https://docs.anthropic.com/

[9] NIST FIPS 180-4. "Secure Hash Standard (SHS)." National Institute of Standards and Technology.

[10] Kshetri, N. & Voas, J. (2018). "Blockchain-Enabled E-Voting." IEEE Software, 35(4), 95-99.